

C-CAT
SECTION B
PREP GUIDE

Operating Systems

Complete Revision Notes

Topics 1 – 9 • Zero to Hero • B.Tech CSE

TABLE OF CONTENTS

01	What is an OS + Booting Process	BIOS, POST, Bootloader, Kernel
02	Process Management	States, PCB, CPU Scheduling Algorithms
03	IPC & Process Synchronization	Shared Memory, Race Condition, Semaphores
04	Deadlock & Handling Methods	Coffman Conditions, Banker's Algorithm
05	Memory Management	Swapping, Paging, Segmentation, Fragmentation
06	Virtual Memory & Demand Paging	Page Fault, Page Replacement, Thrashing
07	System Calls & Dual Mode Operation	User Mode, Kernel Mode, Mode Bit
08	File & Storage Management	File System, Directories, Access Methods
09	Disk Allocation & Disk Scheduling	Contiguous, Linked, Indexed, SCAN, C-SCAN

TOPIC 01

What is an Operating System + Booting Process

WHAT IS AN OS?

An **Operating System (OS)** is a program that manages all hardware (CPU, RAM, disk, keyboard, screen) and acts as a **manager/intermediary** between user programs (apps) and the raw machine.

OS = Hostel Warden Analogy

Students = Programs/Processes

Rooms = Memory (RAM)

Kitchen = CPU

Warden = OS — decides who gets what, when, and prevents conflicts

FIVE MAIN JOBS OF THE OS

- **Process Management** — runs and schedules multiple programs
- **Memory Management** — allocates RAM to processes
- **File Management** — organizes data on disk
- **Device Management** — controls hardware (printer, keyboard, disk)
- **Security/Protection** — keeps processes from interfering with each other

BOOTING PROCESS — STEP BY STEP

Step	What Happens	Key Term
1	Power button pressed — electricity flows	Power ON
2	BIOS/UEFI (firmware on motherboard) runs first	BIOS / UEFI
3	Hardware self-check — RAM, keyboard, disk tested	POST (Power-On Self-Test)
4	Bootloader located on disk and loaded into RAM	Bootloader
5	Bootloader loads the OS Kernel into RAM	Kernel loaded
6	Kernel initialises memory, devices, services	Kernel init
7	Login / Desktop screen appears	OS Ready

Key Definitions:

- **RAM** — Primary memory; fast, temporary; erased on power-off
- **Secondary Memory** — Hard disk/SSD; permanent, slower; OS 'lives' here
- **BIOS/UEFI** — Firmware permanently on motherboard chip; runs before OS loads
- **POST** — Hardware health check; failure = beeps or error screen
- **Bootloader** — Small program with ONE job: find OS on disk and load it into RAM; stops after handing control to kernel
- **Kernel** — Core of the OS; highest privilege; directly controls CPU, memory, hardware
- **Cold Boot** — Starting from powered-off. **Warm Boot** — Restarting without cutting power

■ EXAM KEY POINTS

- OS = resource manager between hardware and user programs
- UEFI is modern replacement for BIOS — supports larger disks, graphical interface
- Bootloader's ONLY job = load kernel. It does NOT keep running after that
- Kernel = highest privilege level; manages CPU/memory/devices directly
- Cold Boot vs Warm Boot — know the difference
- Firmware vs Software — BIOS/UEFI is firmware (on chip, rarely changes)

TOPIC 02

Process Management

Process States · PCB · CPU Scheduling Algorithms

PROCESS — DEFINITION

A **process** = a program currently being executed by the CPU — a program 'in action' with its own memory, status, and resources. (A program sitting idle on disk is NOT a process.)

5 PROCESS STATES

State	Meaning
New	Process is being created by the OS
Ready	Loaded into RAM, waiting for CPU — fully prepared, just needs CPU
Running	CPU is actively executing this process RIGHT NOW
Waiting	Waiting for I/O or external event — CPU won't help even if free
Terminated	Process finished or stopped; being removed from system

State Diagram Flow

New → Ready (process set up, admitted into memory)

Ready → Running (scheduler picks it, CPU allocated)

Running → Waiting (needs I/O — file read, keyboard input, etc.)

Waiting → Ready (I/O completed, back to waiting for CPU)

Running → Ready (time quantum expired OR higher-priority process arrives)

Running → Terminated (process finishes completely)

■ **Ready vs Waiting: Ready = needs ONLY the CPU. Waiting = needs something else (I/O); even giving it the CPU won't help.**

PCB — PROCESS CONTROL BLOCK

The OS creates one **PCB** per process — a data structure storing all information needed to manage and resume that process after a context switch.

- **Process ID (PID)** — unique number identifying the process
- **Process State** — current state (New/Ready/Running/Waiting/Terminated)
- **Program Counter** — address of the NEXT instruction to execute (bookmark)
- **CPU Registers** — values the process was working with mid-execution
- **Memory Information** — which parts of RAM this process uses
- **I/O Status** — devices currently being used by this process

Context Switch = CPU saves current process PCB → loads another process's PCB → switches execution. Pure overhead — no useful work during the switch.

CPU SCHEDULING ALGORITHMS

Algorithm	How it Works	Problem / Downside
FCFS	Processes run in arrival order (queue)	Convoy Effect — long process blocks short ones behind it
SJF	Shortest CPU burst runs first	Starvation — long process may never run if short ones keep arriving
Priority	Highest-priority process runs first	Starvation of low-priority — fixed by Aging (gradually boost priority)

Round Robin	Each process gets fixed time quantum; preempted if not done	Too-small quantum = too much context-switch overhead
-------------	---	--

Key Insight: RR with very large quantum → becomes FCFS

If time quantum > any process's total run time, processes always finish before quantum expires

= No preemption ever triggers = First Come First Served behaviour

■ EXAM KEY POINTS

- Preemptive = OS can forcibly take CPU away (RR, Priority preemptive, SJF preemptive)
- Non-preemptive = process keeps CPU until it finishes or voluntarily gives up (FCFS, non-preemptive SJF)
- Aging = solution to starvation in Priority scheduling
- Round Robin = basis of all time-sharing / multitasking systems
- Turnaround Time = Completion - Arrival | Waiting Time = Turnaround - Burst

TOPIC 03

Inter-Process Communication & Process Synchronization

INTER-PROCESS COMMUNICATION (IPC)

Mechanisms that allow processes to **exchange data** with each other.

Method	How it Works	Speed	Who coordinates?
Shared Memory	OS sets aside a RAM region both processes can read/write directly ^{directly}	Fastest	Processes themselves must synchronise
Message Passing	Processes send/receive messages via the OS (OS acts as postman ^{postman})	Slower	OS handles delivery

RACE CONDITION

A **race condition** occurs when the final result depends on the **timing/order** of concurrent processes accessing shared data — giving incorrect results if the interleaving is unlucky.

Classic Example: counter = counter + 1

CPU does 3 steps: (1) Read counter from memory, (2) Add 1, (3) Write back

If Process A reads '5' and Process B ALSO reads '5' before A writes back '6'...

Both write '6' — final result = 6, but should be 7. One increment LOST!

Fix: enforce mutual exclusion so only ONE process executes this at a time

CRITICAL SECTION & THREE CONDITIONS

Critical Section = the code block where a process accesses shared resources. A correct solution must satisfy ALL THREE:

- **Mutual Exclusion** — Only ONE process in the critical section at a time
- **Progress** — If no one is inside, a waiting process must be allowed in (no unnecessary blocking)
- **Bounded Waiting** — No process should wait FOREVER (anti-starvation guarantee)

SEMAPHORES & MUTEX

Type	Values	Use Case	Operation
Binary Semaphore (Mutex)	0 or 1 only	Single shared resource / lock	wait() acquires, signal() releases
Counting Semaphore	Any integer	Multiple instances of a resource (e.g. 3 printers)	wait() decreases available instances, signal() increases

wait() (P/down) — decreases semaphore; if result < 0, process BLOCKS.

signal() (V/up) — increases semaphore; may wake up a blocked process.

Washroom Lock Analogy

Lock = Binary Semaphore | Entering & locking = wait() | Exiting & unlocking = signal()

Mutual Exclusion: only one person inside at a time

Bounded Waiting: no one should be skipped indefinitely while others cut the queue

■ EXAM KEY POINTS

- Shared Memory = faster, needs manual sync. Message Passing = safer, OS-mediated
- Race condition = timing-dependent incorrect result on shared data
- Critical Section needs: Mutual Exclusion + Progress + Bounded Waiting
- wait() = acquire/block. signal() = release/unblock — direction often tested
- Busy Waiting (spinlock) = wasteful looping; proper semaphores block the process instead
- Producer-Consumer Problem = classic IPC example with bounded buffer

TOPIC 04

Deadlock & Deadlock Handling Methods

WHAT IS A DEADLOCK?

A **deadlock** is a situation where two or more processes are **stuck waiting for each other FOREVER** — each holds a resource the other needs, and none will release until it gets what it's waiting for.

Library Analogy

Student A holds Book 1, waits for Book 2 (held by B)
 Student B holds Book 2, waits for Book 1 (held by A)
 Neither releases → both stuck forever = DEADLOCK

4 COFFMAN CONDITIONS (ALL must hold for deadlock)

Condition	Meaning	Analogy
Mutual Exclusion	Resource used by only one process at a time	Washroom — one person at a time
Hold and Wait	Process HOLDS a resource AND WAITS for another	Student holding Book 1 while waiting for Book 2
No Preemption	Resource cannot be forcibly taken — only voluntarily released	Student gives up their book
Circular Wait	Cycle: A waits for B, B waits for A (or longer chain)	A→B→A or A→B→C→A

■ **Breaking even ONE condition PREVENTS deadlock entirely — this is the key insight!**

4 DEADLOCK HANDLING STRATEGIES

Strategy	When it Acts	How it Works	Example
Prevention	Before deadlock forms (design time)	Ensure at least one Coffman condition can never hold	Request all resources requested upfront (breaks Hold and Wait)
Avoidance	Before granting each request (runtime)	Check if granting keeps system in 'safe state'	Banker's Algorithm
Detection + Recovery	After deadlock occurs	Detect via Resource Allocation Graph cycle, kill or forcibly take a resource	Kill or forcibly take a resource
Ignorance	Never — just ignore it	Assume deadlocks are too rare to handle	Ostrich Algorithm — used by many real OSes!

Resource Allocation Graph (RAG)

Process = Circle | Resource = Rectangle

Process → Resource = 'requesting this resource'

Resource → Process = 'allocated to this process'

CYCLE in the graph = DEADLOCK detected

■ EXAM KEY POINTS

- ALL 4 Coffman conditions must hold simultaneously for deadlock
- Prevention vs Avoidance: Prevention = conditions CAN'T occur. Avoidance = check before granting
- Banker's Algorithm = deadlock AVOIDANCE (not prevention — common confusion!)
- Ostrich Algorithm = ignore deadlocks — surprisingly used in real-world general OSes
- Deadlock vs Starvation: Deadlock = frozen waiting on each other. Starvation = unfair scheduling
- Livelock = processes keep changing state but make no real progress (not frozen, but stuck)
- Safe State = OS can guarantee all processes finish eventually. Unsafe ≠ deadlock immediately

TOPIC 05

Memory Management

Swapping · Paging · Segmentation · Fragmentation

WHY MEMORY MANAGEMENT?

RAM is **limited and fast**. Many processes need it simultaneously. The OS must allocate, track, and free RAM efficiently — while protecting each process's memory from others.

SWAPPING

Swapping = moving an entire process temporarily **OUT of RAM onto disk** (swap space) to free RAM, then bringing it back when needed.

- Used when RAM is full and another process needs to run
- Slow — disk access is much slower than RAM
- Swap space = reserved area on disk specifically for this purpose

PAGING

Paging = divide process memory into fixed-size **Pages** AND divide RAM into same-size **Frames**. Any page can go into any free frame — no contiguous space needed.

Term	Definition
Page	Fixed-size block of a PROCESS's memory (e.g. 4 KB)
Frame	Fixed-size block of RAM — same size as a page
Page Table	Per-process lookup table: Page Number → Frame Number

Paging solves External Fragmentation

Old contiguous allocation: need one big free block → scattered free blocks can't be used

Paging: any free frame works for any page → no wasted scattered space

BUT introduces Internal Fragmentation: last page may not fill its frame completely

SEGMENTATION

Segmentation = divide process memory by **logical meaning** (Code, Data, Stack segments) — variable sizes, not fixed.

- Segment Table stores: Base Address (where segment starts in RAM) + Length
- More meaningful to programmer than paging — segments correspond to actual program parts
- Downside: variable sizes → External Fragmentation (same problem as old contiguous allocation)

FRAGMENTATION — CRITICAL COMPARISON

Type	Where Waste Occurs	Caused By	Solved By
External Fragmentation	Between allocated blocks — free space scattered in small gaps	Contiguous allocation	Paging
Internal Fragmentation	Inside an allocated block — block not fully used	Paging (fixed block size)	Smaller page size (trade-off)

■ EXAM KEY POINTS

- Page = piece of process | Frame = piece of RAM | Same fixed size
- External Fragmentation: free space exists but too scattered (contiguous/segmentation problem)
- Internal Fragmentation: wasted space INSIDE a block (paging problem)
- Paging SOLVES external but INTRODUCES internal fragmentation
- Segmentation = variable-size, logical units (Code/Data/Stack) with Segment Table
- Real systems often combine: Paged Segmentation = logical segments + paged within each

TOPIC 06

Virtual Memory, Demand Paging & Thrashing

VIRTUAL MEMORY

Virtual Memory = a technique that lets a process run even if it's **larger than available RAM**, by keeping only currently-needed pages in RAM and the rest on disk — giving each process the **illusion** of a large, private memory space.

Study Desk Analogy

Library (all books) = process's total memory | Your desk = RAM

You don't bring ALL books to your desk — only what you're currently reading

When you need a different book, fetch it from the shelf (disk)

To make room, put back a book you're not using

DEMAND PAGING

Demand Paging = pages are loaded into RAM **only when the process actually accesses them** — NOT pre-loaded in advance.

Step	Action
1	Process starts — most pages NOT in RAM (page table entries point to disk)
2	CPU tries to access a page
3	Page Table checked — is this page in RAM?
4 — Page HIT	Page IS in RAM → access proceeds normally (fast)
5 — Page FAULT	Page NOT in RAM → OS must fetch from disk (slow!)
6	Process moves to WAITING state (I/O request — connects to Topic 2 states!)
7	OS finds a free frame, fetches page from disk into that frame
8	Page Table updated. Process → Ready → Running. Instruction retried

PAGE REPLACEMENT ALGORITHMS

When a Page Fault occurs and RAM has NO free frames, the OS must **evict one page** to make room:

Algorithm	Eviction Rule	Key Note
FIFO	Remove the page that has been in RAM the LONGEST	Simple but ignores usage — may remove frequently-used pages (poor)
LRU	Remove the page NOT USED for the longest time	Better — recently used pages likely needed again soon
Optimal	Remove page not needed for the longest time IN THE FUTURE	Theoretical best — impossible in practice (requires knowing the future)

■ **Belady's Anomaly: With FIFO, adding MORE frames can sometimes INCREASE page faults — counterintuitive! LRU does not suffer from this.**

THRASHING

Thrashing = system spends most of its time **handling page faults** (swapping pages in/out of disk) and almost no time executing actual process instructions — CPU utilization DROPS despite disk working overtime.

Why Thrashing Happens & How to Fix

Cause: Too many processes → each gets too few frames → constant page faults

Counter-intuitive trap: Low CPU usage → admin adds MORE processes → makes thrashing WORSE

Correct fix: REDUCE number of running processes (so each gets more frames) OR add more RAM

During thrashing: processes mostly in WAITING state (waiting for page fault resolution)

■ EXAM KEY POINTS

- Virtual Memory = illusion of large memory using RAM + disk together
- Demand Paging = load page only when accessed (not in advance)
- Page Fault = page not in RAM → triggers disk fetch → process goes to WAITING state
- FIFO: oldest-in evicted. LRU: least recently used evicted. Optimal: theoretical only
- Belady's Anomaly = FIFO only — more frames can mean MORE page faults
- Thrashing: CPU utilization DOWN, disk activity UP, barely any execution — fix by reducing processes
- Effective Access Time = $(1-p) \times \text{memory_time} + p \times \text{disk_time}$ (p = page fault rate)

TOPIC 07

System Calls, Dual Mode Operation, Kernel & User Mode

WHY DUAL MODE?

If every program had unrestricted hardware access, one buggy or malicious app could crash the entire system, corrupt files, or overwrite other processes' memory. Dual Mode prevents this.

USER MODE vs KERNEL MODE

Aspect	User Mode (Mode Bit = 1)	Kernel Mode (Mode Bit = 0)
Who runs here?	Regular apps — browser, games, editor	OS Kernel
Hardware access?	NO — restricted	YES — full, direct access
Privileged instructions?	BLOCKED — causes error/trap if attempted	Allowed
Memory access?	Limited to own process space	Any memory location

Mode Bit

A single hardware bit inside the CPU indicating current mode

Mode Bit = 1 → User Mode | Mode Bit = 0 → Kernel Mode

CPU checks mode bit BEFORE executing any privileged instruction

If User Mode + privileged instruction attempted → CPU BLOCKS IT and raises a trap/error

SYSTEM CALL — THE ONLY GATEWAY TO KERNEL

A **system call** = the controlled mechanism by which a User Mode program requests the kernel to perform a privileged operation on its behalf.

Step	What Happens
1	User Mode program needs privileged operation (e.g., read a file)
2	Program makes a system call (special instruction — also called a 'trap')
3	MODE SWITCH: Mode Bit flips 1 → 0 (User → Kernel Mode)
4	Kernel performs the privileged operation (disk read, memory allocation, etc.)
5	MODE SWITCH: Mode Bit flips 0 → 1 (Kernel → User Mode)
6	Result returned to program; program resumes in User Mode

SYSTEM CALL CATEGORIES

- **Process Control** — create, terminate processes (connects to Topic 2's process states)
- **File Management** — open, read, write, close files
- **Device Management** — request/release devices, read/write devices
- **Information Maintenance** — get/set system time, process info
- **Communication** — send/receive messages between processes (connects to Topic 3 IPC)

Connection to Topics 1, 2, 6

Topic 1: Kernel (from booting) = the thing that runs in Kernel Mode

Topic 2: When process needs I/O → system call → kernel handles it → process to Waiting state

Topic 6: Page Fault → hardware auto-triggers mode switch to Kernel Mode (not a deliberate system call!)

Trap = software-generated mode switch (system calls, errors). Interrupt = hardware-generated (I/O done, notify CPU)

■ EXAM KEY POINTS

- Dual Mode = User Mode (restricted) + Kernel Mode (full privilege) — protects system
- Mode Bit: 0 = Kernel, 1 = User (values may vary but concept is fixed)
- System Call = ONLY controlled way for User Mode to request kernel operations
- Attempted privileged instruction in User Mode → CPU blocks it and raises trap
- Page Fault = hardware-triggered mode switch (automatic, not deliberate)
- Trap = software-triggered | Interrupt = hardware-triggered (both cause Kernel Mode switch)
- Mode switching has overhead (like context switching) — excessive system calls slow programs

TOPIC 08

File & Storage Management — File System Structure

FILE & FILE SYSTEM — DEFINITIONS

File = a named collection of related data stored on secondary memory (disk) — permanent storage even after power-off.

File System = the OS component that manages how files are named, stored, organized, and retrieved on disk. It is the 'catalog', not the content itself.

FILE ATTRIBUTES (METADATA)

Metadata = data ABOUT a file (not the file's content itself).

- **Name** — how users identify the file
- **Size** — amount of data
- **Type** — kind of file (often via extension: .txt, .exe, .jpg)
- **Location** — where on disk the data physically lives
- **Permissions** — who can Read (R), Write (W), Execute (X)
- **Timestamps** — created, last modified, last accessed

DIRECTORY STRUCTURES

Directory = a special file that contains a LIST of other files/directories + pointers to where they are. It does NOT hold user data directly.

Structure	Description	Problem
Single-Level	All files in one flat list, no folders	No two files can have the same name; chaotic with many files
Two-Level	Each USER gets their own directory	Solves naming conflict between users; different users can have same file
Tree-Structured	Directories contain files AND sub-directories (most common)	Most significant — this is what all modern OSes use

Path = route through the directory tree

Example: /Users/Alice/Documents/report.docx

Each / separates one level of the tree from the next

From root (/) → Users folder → Alice's folder → Documents folder → file

FILE ACCESS METHODS

Method	How it Works	Best For
Sequential Access	Read data in order from start to end — cannot skip ahead	Simple log files; reading entire file
Direct (Random) Access	Jump directly to any location using position/offset	Databases; reading last paragraph without scanning entire file

DISK BLOCKS — BRIDGE TO TOPIC 9

A file's actual data is stored in **disk blocks** — fixed-size chunks of disk space (the basic unit of disk read/write). Files spanning multiple blocks need a system to track which blocks belong to which file — this is Topic 9.

■ EXAM KEY POINTS

- File = data on disk. File System = catalog/organizer (not the data itself)
- Metadata = data ABOUT a file (name, size, type, permissions, timestamps)
- Directory = stores LIST of files + pointers, NOT actual user data
- Single-Level (no folders) → Two-Level (per-user) → Tree-Structured (folders in folders) — know the evolution
- Sequential = must read in order. Direct/Random = jump anywhere — use Direct for 'find last paragraph'
- Mounting = attaching an external disk's file system into the directory tree
- Permissions: R (read), W (write), X (execute) — enforced by kernel

TOPIC 09

Disk Space Allocation & Disk Scheduling Algorithms

PART A: DISK SPACE ALLOCATION

How does the OS decide where a file's **disk blocks** go, and how does it track them?

Method	How It Works	Random Access?	Ext. Fragmentation?	Key Weakness
Contiguous	All blocks stored consecutively on disk	YES — easy: start + offset	YES — problem!	External Fragmentation as files created/deleted
Linked	Each block has pointer to next block (chain)	NO — must follow chain from block 1	NO	No random access; pointer corruption loses file
Indexed	Separate index block lists ALL block locations	YES — look up index directly	NO	Index block uses extra space; large files may be slow

Pattern Across Topics — Fragmentation Recurring Theme

Topic 5: Contiguous Memory Allocation → External Fragmentation (solved by Paging)

Topic 9: Contiguous Disk Allocation → External Fragmentation (solved by Linked/Indexed)

Topic 5: Paging → Internal Fragmentation | Topic 9: Linked → forced sequential only

Recurring pattern: contiguous = simple + fast + fragmentation problem

PART B: DISK SCHEDULING ALGORITHMS

Seek Time = time for disk's read/write head to physically move to the requested track. Disk scheduling minimizes total seek time across multiple pending requests.

Setup for all examples: disk tracks 0–199, head at position 50, requests: 98, 183, 37, 122, 14, 124, 65, 67

Algorithm	Head Movement Rule	Advantage	Disadvantage
FCFS	Service in arrival order	Simple, fair in arrival order	Erratic head movement — huge seek time (similar to SJF starvation)
SSTF	Service request CLOSEST to current head position	Minimizes average seek time	Starvation of far requests (similar to SJF starvation)
SCAN	Move in one direction, service all requests, reach end, REVERSE direction	No starvation	Farther end of disk waits longer on reverse trip
C-SCAN	Like SCAN, but on reaching end, JUMP back to start (no service times across)	Uniform service times across disk	Slight sweep again from non-serving return jump

SCAN = Elevator Algorithm

Like a building elevator: goes up serving all floor requests, reaches top, comes back down serving requests

C-SCAN: elevator goes up serving requests, reaches top, teleports to ground floor, sweeps up again

C-SCAN is fairer because tracks near the 'start' don't wait for a full down+up cycle

■ EXAM KEY POINTS

- Contiguous = fast + random access BUT external fragmentation
- Linked = no fragmentation BUT NO random access (must follow chain from block 1)
- Indexed = random access + no fragmentation BUT extra index block space needed
- Seek Time = time for head to MOVE to track — what scheduling algorithms minimize
- FCFS (disk) = arrival order, erratic. SSTF = closest first, can starve. SCAN = elevator. C-SCAN = uniform
- SSTF is to disk scheduling what SJF is to CPU scheduling (similar starvation risk)
- FCFS is to disk scheduling what FCFS is to CPU scheduling (same name, same 'order of arrival' idea)

MASTER CHEATSHEET — ALL 9 TOPICS AT A GLANCE

Topic	Must-Know Concept	Common Exam Trap
01 OS & Boot	POST → Bootloader → Kernel. Kernel = highest privilege	Bootloader stops after loading kernel (doesn't keep running)
02 Processes	5 States: New/Ready/Running/Waiting/Terminated. PCB per process	Ready ≠ Waiting. Ready = needs CPU. Waiting = needs I/O
02 Scheduling	FCFS→Convoy. SJF→Starvation. RR→time quantum. Aging fixes starvation	RR with huge quantum = FCFS behaviour
03 IPC	Shared Memory (fast). Message Passing (safe). Race Condition = timing bug	wait() = acquire/block. signal() = release. Not the other way!
03 Sync	Critical Section needs: Mutual Exclusion + Progress + Bounded Waiting	Bounded Waiting ≠ Mutual Exclusion. BW = anti-starvation guarantee
04 Deadlock	4 Coffman conditions. Break ONE = no deadlock	Prevention vs Avoidance: Prevention = design time. Avoidance = runtime
04 Handling	Banker's Algorithm = Avoidance. Ostrich = Ignorance	Banker's = Avoidance NOT Prevention (most confused pair!)
05 Memory	Paging: Page=process piece, Frame=RAM piece, Page Table maps them	External frag = scattered free space. Internal frag = wasted space
06 Virtual Mem	Demand Paging: load page only when accessed. Page Fault → Waiting	Bellare's Anomaly = FIFO only. More frames can mean MORE faults
06 Thrashing	Too many processes → too few frames → constant page faults → CPU idle	File REDUCE processes (not add more, despite low CPU appt)
07 Dual Mode	User Mode (restricted) ↔ Kernel Mode (full). Mode Bit switches them	Page Fault = hardware-triggered mode switch (not a deliberate switch)
08 File System	Directory = list of pointers, not data. Tree structure = most common	Metadata ≠ file content. Direct Access = jump anywhere. Sequential Access = jump sequentially
09 Disk Alloc	Contiguous=fast+fragmentation. Linked=no random access. Indexed=best of all	best of all allocation = sequential access ONLY (cannot jump to random)
09 Disk Sched	SCAN=elevator. C-SCAN=uniform sweep. SSTF=closest first (can starve)	SSTF starvation mirrors SJF starvation. FCFS disk mirrors FCFS CPU

You have covered Operating Systems from zero to hero — 9 complete chapters, fully exam-ready.

Next: Phase 2 — Computer Networks (Topics 10–13). Keep the momentum going!